

Windows Server 2025 Cybersecurity Home Lab

1. Project Overview

This project demonstrates the installation, configuration, and basic security administration of **Windows Server 2025** in a virtualized home lab environment.

The server was deployed using **Oracle VirtualBox** and configured as a standalone Windows Server in a `WORKGROUP` environment. PowerShell was used extensively to perform system administration and security-related tasks.

The purpose of this lab was to gain practical experience with:

- Virtualization
- Windows Server administration
- PowerShell
- Local user management
- Administrator privileges
- Windows Firewall
- Firewall rules
- Windows Services
- Running processes
- Windows Update inspection
- Event Logs
- Basic network testing
- Server directory organization
- Security monitoring and troubleshooting

This project was also documented as a portfolio project to demonstrate practical cybersecurity and system administration skills.

2. Lab Objectives

The main objectives of this project were:

1. Install Windows Server 2025 in VirtualBox.
2. Configure the server and verify the operating system.
3. Configure a local administrator account.
4. Create and manage another local administrator account.
5. Inspect Windows Firewall profiles and rules.

6. Safely test a firewall rule.
7. Create and remove a custom outbound firewall rule.
8. Inspect and control Windows Services.
9. Monitor running processes.
10. Inspect Windows Update configuration.
11. Examine Windows Event Logs.
12. Create an organized laboratory directory structure.
13. Document the work for a cybersecurity portfolio.

3. Software and Lab Environment

Component	Configuration
Hypervisor	Oracle VirtualBox
Operating System	Windows Server 2025 Standard Evaluation
Server Name	WINSERVER
Username	shah_server
Additional Administrator	LabAdmin
Network Mode	NAT
IPv4 Address	10.0.2.15
Subnet Mask	255.255.255.0
Default Gateway	10.0.2.2
Workgroup	WORKGROUP
Virtual Disk	20 GB
Remote Management	Enabled
Remote Desktop	Disabled
PowerShell	Used for administration

4. Downloading Windows Server 2025

Windows Server 2025 was obtained as an official evaluation ISO from Microsoft.

The downloaded ISO was approximately **7.59 GB**.

The ISO file was stored on the host computer before creating the virtual machine.

ISO purpose

An ISO file is a virtual copy of an installation DVD.

Instead of inserting a physical Windows Server DVD into a computer, VirtualBox can mount the ISO file as a virtual optical disk and use it to install Windows Server.

5. Preparing Oracle VirtualBox

Oracle VirtualBox was used as the virtualization platform.

A new virtual machine was created specifically for the Windows Server lab.

Virtual machine configuration

The VM was configured with:

- Name: WinServer-Lab / Windows Server 2025 lab VM
- Operating System: Microsoft Windows
- Version: Windows Server 2025 (64-bit)
- RAM: suitable amount for the available host memory
- CPU: allocated virtual processor(s)
- Virtual Disk: 20 GB
- Network Adapter: NAT
- Installation Media: Windows Server 2025 ISO

Why NAT was selected

NAT allows the virtual machine to access the Internet through the host computer while keeping the lab environment relatively isolated from the physical network.

This was useful for:

- Windows updates
 - Network testing
 - Internet connectivity testing
-

6. Installing Windows Server 2025

The Windows Server 2025 ISO was attached to the virtual machine's virtual optical drive.

The VM was then started.

The Windows installation process was followed.

During installation, the appropriate Windows Server 2025 Standard Evaluation edition was selected.

A local administrator password was configured.

After installation, Windows Server started successfully.

7. Initial Server Configuration

After installation, the Windows Server configuration was checked using the Server Configuration interface and PowerShell.

The following configuration was verified:

- Server name
- Workgroup
- Network configuration
- Remote management
- Remote Desktop
- Windows Update settings

The server was kept in:

WORKGROUP

rather than joining an Active Directory domain because this project focused on standalone Windows Server administration.

8. Server Name

The server was configured with the computer name:

WINSERVER

The server name can be checked using:

`hostname`

Another way to verify the computer name is:

`Get-ComputerInfo | Select-Object CsName`

The server configuration interface also displayed:

```
Computer name: WINSERVER
```

Why the server name matters

A hostname identifies a computer on a network.

For example:

```
WINSERVER
```

is easier to identify than using only an IP address.

9. Checking the Windows Server Version

The installed Windows Server version was verified with:

```
Get-ComputerInfo | Select-Object WindowsProductName, WindowsVersion,  
OsBuildNumber
```

The result showed:

```
Windows Server 2025 Standard Evaluation
```

with the corresponding Windows version and OS build.

Command explanation

```
Get-ComputerInfo
```

Retrieves detailed information about the Windows computer.

```
Select-Object
```

Selects only the information we are interested in displaying.

The command was used to avoid displaying hundreds of unnecessary system properties.

10. Checking the Current User

The current Windows account was checked with:

```
whoami
```

The result was:

```
winserver\shah_server
```

This confirms the current account and computer/domain context.

The format is:

```
COMPUTER_OR_DOMAIN\USERNAME
```

Therefore:

```
winserver\shah_server
```

means the user account `shah_server` is being used on the local Windows Server system.

11. Network Configuration

The Windows Server VM used NAT networking.

The server received the following IPv4 configuration:

```
IPv4 Address : 10.0.2.15  
Subnet Mask  : 255.255.255.0  
Gateway      : 10.0.2.2
```

Internet connectivity was tested with:

```
ping 8.8.8.8
```

The test successfully returned replies.

Another connectivity test was performed using:

```
ping 1.1.1.1
```

The test also successfully returned replies after the temporary firewall experiment was removed.

12. Windows Event Log Inspection

Windows Event Logs provide information about system activity, errors, warnings, security-related events, and other operating system activity.

A sample of recent System events was viewed using:

```
Get-WinEvent -LogName System -MaxEvents 10
```

Explanation

```
Get-WinEvent
```

Retrieves Windows event log entries.

```
-LogName System
```

Specifies the System event log.

```
-MaxEvents 10
```

Limits the output to the most recent 10 events.

Security relevance

Event Logs are important because administrators and security analysts can use them to investigate:

- System errors
- Service failures
- Startup problems
- Configuration changes
- Security-related activity
- Troubleshooting information

13. Local User Administration

The local Windows accounts were inspected using:

```
Get-LocalUser
```

The system contained accounts including:

```
Administrator
DefaultAccount
Guest
shah_server
WDAGUtilityAccount
```

The existing administrator account was also verified.

14. Creating a New Local Administrator

A second local account named:

```
LabAdmin
```

was created for the laboratory.

The command used was:

```
New-LocalUser -Name "LabAdmin" -Password (Read-Host -AsSecureString "Enter
Password")
```

The password was entered securely when prompted.

The account was then verified:

```
Get-LocalUser -Name "LabAdmin"
```

15. Adding LabAdmin to the Administrators Group

The new account was added to the local Administrators group:

```
Add-LocalGroupMember -Group "Administrators" -Member "LabAdmin"
```

The group membership was then verified:

```
Get-LocalGroupMember -Group "Administrators"
```

The Administrators group contained accounts including:

```
Administrator
LabAdmin
shah_server
```

Security concept

Administrator accounts have elevated privileges.

Administrative privileges should be given only when necessary because an administrator account can make significant changes to the operating system.

16. Windows Firewall

Windows Firewall was inspected using PowerShell.

The following command was used:

```
Get-NetFirewallProfile | Select-Object Name, Enabled, DefaultInboundAction, DefaultOutboundAction
```

The three standard firewall profiles were examined:

- Domain
- Private
- Public

The firewall was enabled for the profiles.

17. Understanding Inbound and Outbound Traffic

Inbound traffic

Inbound traffic is traffic coming **into** the server.

Example:

```
Another computer → Windows Server
```

Outbound traffic

Outbound traffic is traffic leaving the server.

Example:

Windows Server → Internet

Firewall rules can control both directions.

18. Inspecting Firewall Rules

Inbound firewall rules were inspected using:

```
Get-NetFirewallRule -Direction Inbound | Select-Object -First 10 DisplayName, Enabled, Action
```

Outbound rules were inspected using:

```
Get-NetFirewallRule -Direction Outbound | Select-Object -First 10 DisplayName, Enabled, Action
```

The important properties were:

DisplayName

The name of the firewall rule.

Enabled

Indicates whether the rule is currently active.

Action

Specifies what happens when the rule matches traffic.

Typical actions include:

```
Allow  
Block
```

19. Testing an Existing Firewall Rule

An existing Windows Firewall rule was selected for a controlled laboratory test:

```
Remote Event Log Management (RPC)
```

The rule was inspected using:

```
Get-NetFirewallRule -DisplayName "Remote Event Log Management (RPC)" |  
Select-Object DisplayName, Enabled, Direction, Action
```

The rule was temporarily enabled:

```
Enable-NetFirewallRule -DisplayName "Remote Event Log Management (RPC)"
```

The status was checked.

The rule was then disabled:

```
Disable-NetFirewallRule -DisplayName "Remote Event Log Management (RPC)"
```

This demonstrated how PowerShell can be used to manage individual Windows Firewall rules.

20. Creating a Custom Firewall Rule

A custom firewall rule was created for a controlled security experiment.

The objective was to block outbound ICMP Echo Requests to:

1.1.1.1

The following rule was created:

```
New-NetFirewallRule `  
-DisplayName "Lab Block Ping 1.1.1.1" `  
-Direction Outbound `  
-Protocol ICMPv4 `  
-IcmpType 8 `  
-RemoteAddress 1.1.1.1 `  
-Action Block
```

Explanation

```
New-NetFirewallRule
```

Creates a new Windows Firewall rule.

```
-DisplayName
```

Gives the rule a readable name.

```
-Direction Outbound
```

The rule controls traffic leaving the server.

```
-Protocol ICMPv4
```

The rule applies to IPv4 ICMP traffic.

```
-IcmpType 8
```

ICMP type 8 is an Echo Request, which is used by the normal IPv4 ping operation.

```
-RemoteAddress 1.1.1.1
```

The rule applies specifically to the destination address 1.1.1.1.

```
-Action Block
```

The matching traffic is blocked.

21. Verifying the Custom Firewall Rule

The rule was verified with:

```
Get-NetFirewallRule -DisplayName "Lab Block Ping 1.1.1.1" | Select-Object  
DisplayName, Enabled, Direction, Action
```

The output showed:

```
DisplayName : Lab Block Ping 1.1.1.1  
Enabled     : True  
Direction  : Outbound  
Action      : Block
```

A ping test was then performed:

```
ping 1.1.1.1
```

The request failed because the firewall rule blocked the outbound ICMP traffic.

The result showed:

```
General failure
```

This demonstrated that the firewall rule was working as intended.

22. Removing the Test Firewall Rule

The temporary laboratory rule was removed after testing:

```
Remove-NetFirewallRule -DisplayName "Lab Block Ping 1.1.1.1"
```

Connectivity was tested again:

```
ping 1.1.1.1
```

The server successfully received replies again.

This confirmed that the temporary rule had been removed and normal connectivity restored.

23. Windows Services

Windows Services were inspected using:

```
Get-Service | Select-Object -First 15 Name, Status, StartType
```

The important properties were:

Name

The internal service name.

Status

The current state of the service.

Examples:

```
Running  
Stopped
```

StartType

Determines how Windows starts the service.

Examples include:

```
Automatic  
Manual
```

Disabled

A service being stopped does not automatically mean there is a problem. Some services are designed to run only when required.

24. Testing the Background Intelligent Transfer Service

The Background Intelligent Transfer Service (BITS) was selected for a safe service-management exercise.

The service was checked:

```
Get-Service -Name BITS
```

Detailed information was displayed using:

```
Get-Service -Name BITS | Select-Object Name, DisplayName, Status, StartType
```

The service was started:

```
Start-Service -Name BITS
```

The status was verified:

```
Get-Service -Name BITS
```

The service was then stopped:

```
Stop-Service -Name BITS
```

Finally, its status was checked again.

This demonstrated basic Windows Service administration without modifying critical system services.

25. Running Process Monitoring

Running processes were inspected using:

```
Get-Process | Select-Object -First 15 Name, Id, CPU
```

Important properties include:

Name

The process name.

Id

The Process ID (PID).

CPU

The amount of processor time associated with the process.

26. Checking a Specific Process

PowerShell itself was checked with:

```
Get-Process -Name powershell
```

This demonstrates how administrators can inspect a specific running process.

27. Finding Processes Using the Most CPU

Processes were sorted by CPU usage:

```
Get-Process | Sort-Object CPU -Descending | Select-Object -First 10 Name, Id, CPU
```

Explanation

```
Sort-Object CPU
```

Sorts processes based on CPU usage.

```
-Descending
```

Places the highest values first.

```
Select-Object -First 10
```

Displays only the first 10 results.

This provides a quick way to identify processes consuming significant CPU time.

28. Checking Windows Explorer Safely

The following command was also tested:

```
Get-Process -Name explorer -ErrorAction SilentlyContinue
```

`-ErrorAction SilentlyContinue` prevents PowerShell from displaying an error if the process does not exist.

No output was expected in this Server Core environment because the traditional Explorer graphical shell is not running in the same way as on a normal Windows desktop.

29. Windows Update Inspection

Because this lab was running on a limited 20 GB virtual disk, large Windows updates were not intentionally installed during the lab.

Instead, the Windows Update service and available update-related commands were inspected.

The Windows Update service was checked:

```
Get-Service -Name wuauserv
```

Detailed information was checked:

```
Get-Service -Name wuauserv | Select-Object Name, Status, StartType
```

The available Windows Update-related commands were searched with:

```
Get-Command *WindowsUpdate*
```

This demonstrated how PowerShell can be used to investigate Windows update functionality without unnecessarily consuming the limited virtual disk space.

30. Lab Directory Structure

A dedicated directory was created inside the Windows Server VM:

```
C:\Lab
```

Three subdirectories were created:

```
C:\Lab
├── Logs
├── Scripts
└── Tools
```

The main directory was created with:

```
New-Item -Path "C:\Lab" -ItemType Directory
```

The subdirectories were created with:

```
New-Item -Path "C:\Lab\Logs","C:\Lab\Scripts","C:\Lab\Tools" -ItemType
Directory
```

The directory structure was checked:

```
Get-ChildItem "C:\Lab"
```

31. Checking Directory Permissions

The owner of the laboratory directory was checked using:

```
Get-Acl "C:\Lab" | Select-Object Path, Owner
```

The owner was:

```
BUILTIN\Administrators
```

This demonstrated basic Windows access-control inspection.

32. Important PowerShell Concepts Learned

Several PowerShell concepts were practiced throughout the lab.

Pipeline

The pipe character:

|

passes the output of one command to another command.

Example:

```
Get-Process | Select-Object Name, Id
```

Select-Object

Used to select specific properties:

```
Select-Object Name, Status
```

Sort-Object

Used to sort results:

```
Sort-Object CPU -Descending
```

-First

Limits the number of results:

```
-First 10
```

-ErrorAction

Controls how PowerShell handles errors.

Example:

```
-ErrorAction SilentlyContinue
```

33. Security Lessons Learned

This lab provided practical experience with several important cybersecurity concepts.

Principle of Least Privilege

Administrative privileges should be provided only when required.

Firewall Management

Firewalls control network traffic according to rules and conditions.

Inbound vs Outbound Security

Security controls must consider both incoming and outgoing traffic.

Process Monitoring

Running processes can provide useful information when investigating system behavior.

Service Management

Unexpected services or service changes can be important during security investigations.

Event Logs

Logs provide evidence that can help identify system problems and security events.

User Management

Local accounts and group memberships determine who can perform administrative actions.

Network Troubleshooting

Ping can help determine whether network communication is working.

34. Final Lab Result

The Windows Server 2025 home lab was successfully installed and configured in Oracle VirtualBox.

The project demonstrated practical administration of a Windows Server environment using PowerShell.

The completed tasks included:

- Windows Server installation
- Server identification
- System information verification
- Network connectivity testing
- Event Log inspection
- Local user management
- Administrator group management
- Windows Firewall inspection
- Firewall rule management
- Custom firewall rule creation
- Firewall troubleshooting
- Windows Service management
- Process monitoring
- Windows Update inspection
- Directory and permission management

The lab provides a foundation for more advanced cybersecurity activities such as:

- Active Directory
- Windows security auditing
- Event log analysis
- Authentication monitoring
- PowerShell security automation
- Network security monitoring
- Vulnerability assessment
- Incident investigation

35. Demonstration / Screenshot Evidence

The screenshots provide visual evidence that the laboratory tasks were actually performed.

Screenshot 1 — Windows Server Information

Command:

```
Get-ComputerInfo | Select-Object WindowsProductName, WindowsVersion,  
OsBuildNumber
```

Purpose:

Demonstrates that Windows Server 2025 was successfully installed.

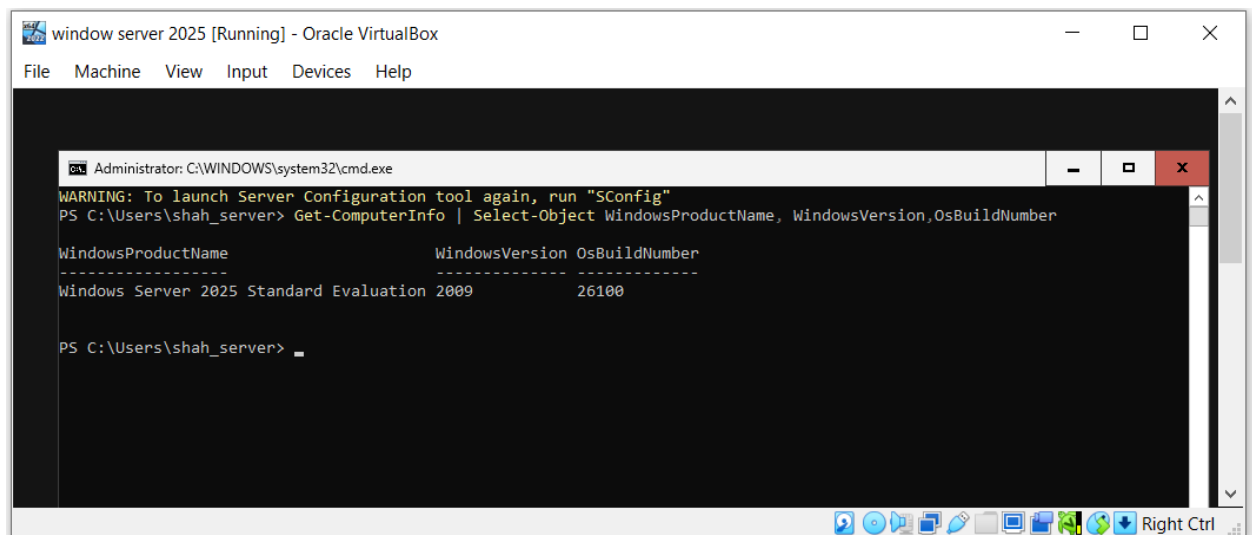


Figure 1. Windows Server 2025 system information

Screenshot 2 — Local Administrators

Command:

```
Get-LocalGroupMember -Group "Administrators"
```

Purpose:

Demonstrates local administrator group membership and the creation of the `LabAdmin` account.

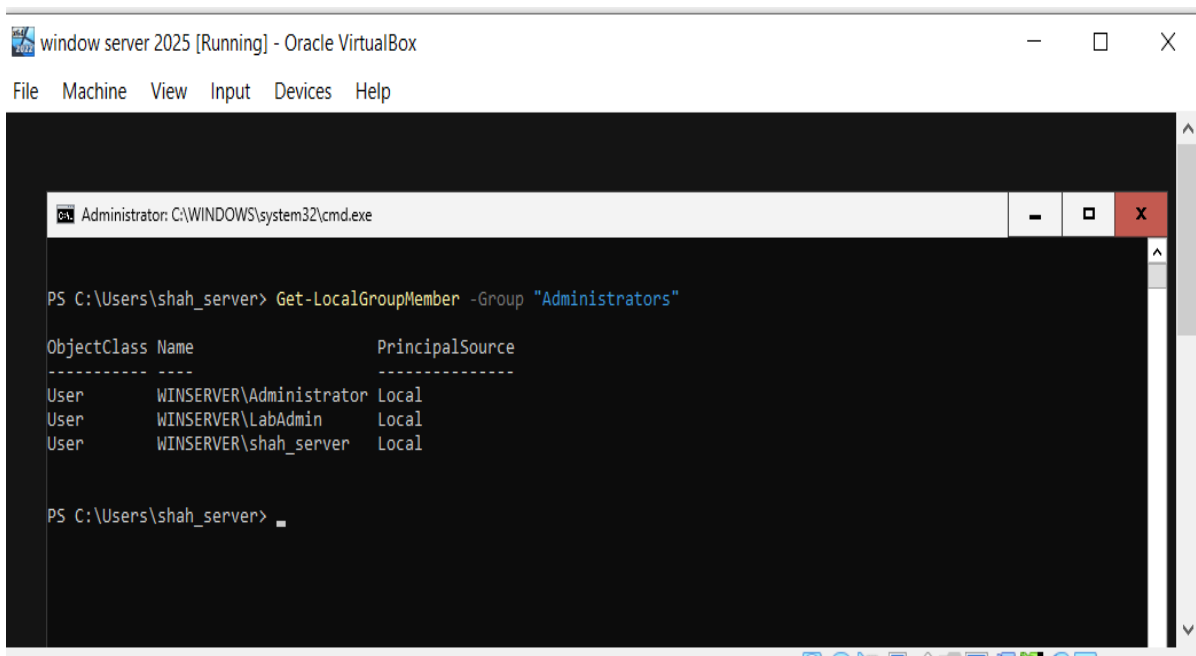


Figure 2. Local Administrators group

Screenshot 3 — Windows Firewall Status

Command:

```
Get-NetFirewallProfile | Select-Object Name, Enabled, DefaultInboundAction, DefaultOutboundAction
```

Purpose:

Demonstrates that Windows Firewall profiles were inspected and enabled.

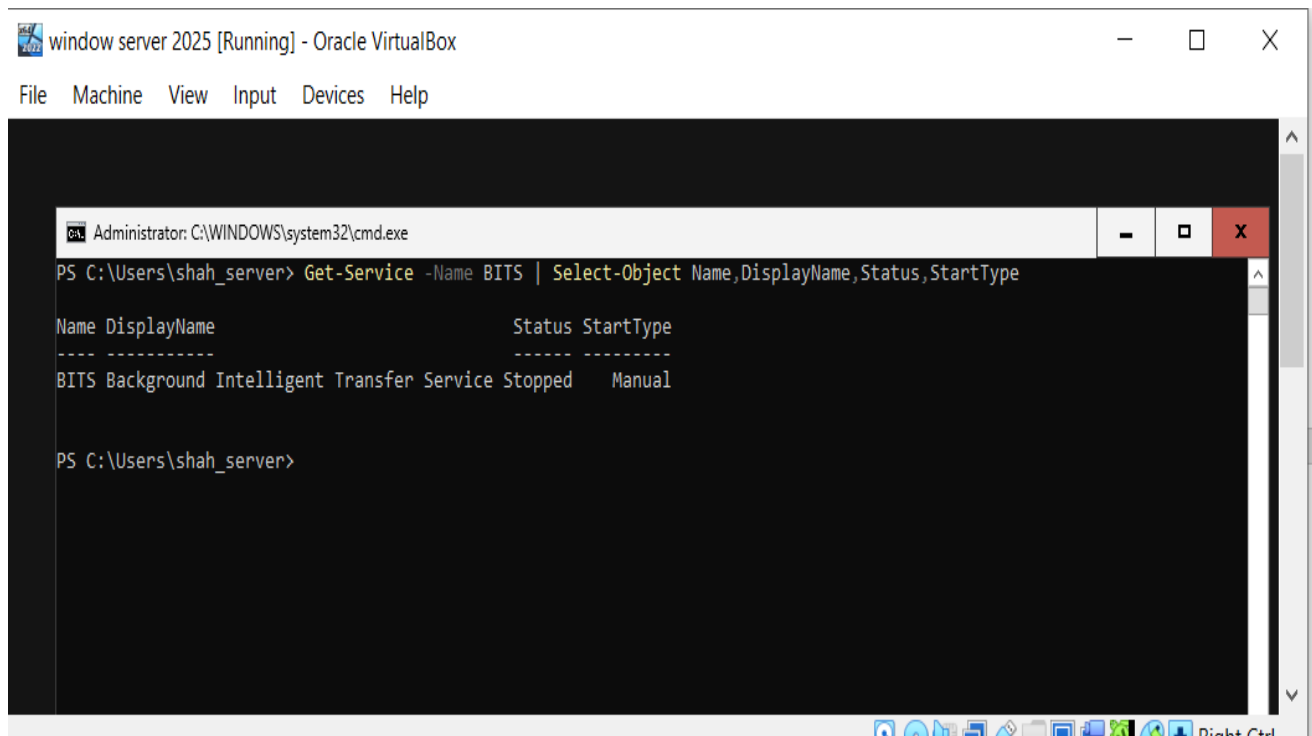


Figure 3. Windows Firewall profile status

Screenshot 4 — Windows Service

Command:

```
Get-Service -Name BITS | Select-Object Name, DisplayName, Status, StartType
```

Purpose:

Demonstrates Windows Service inspection.

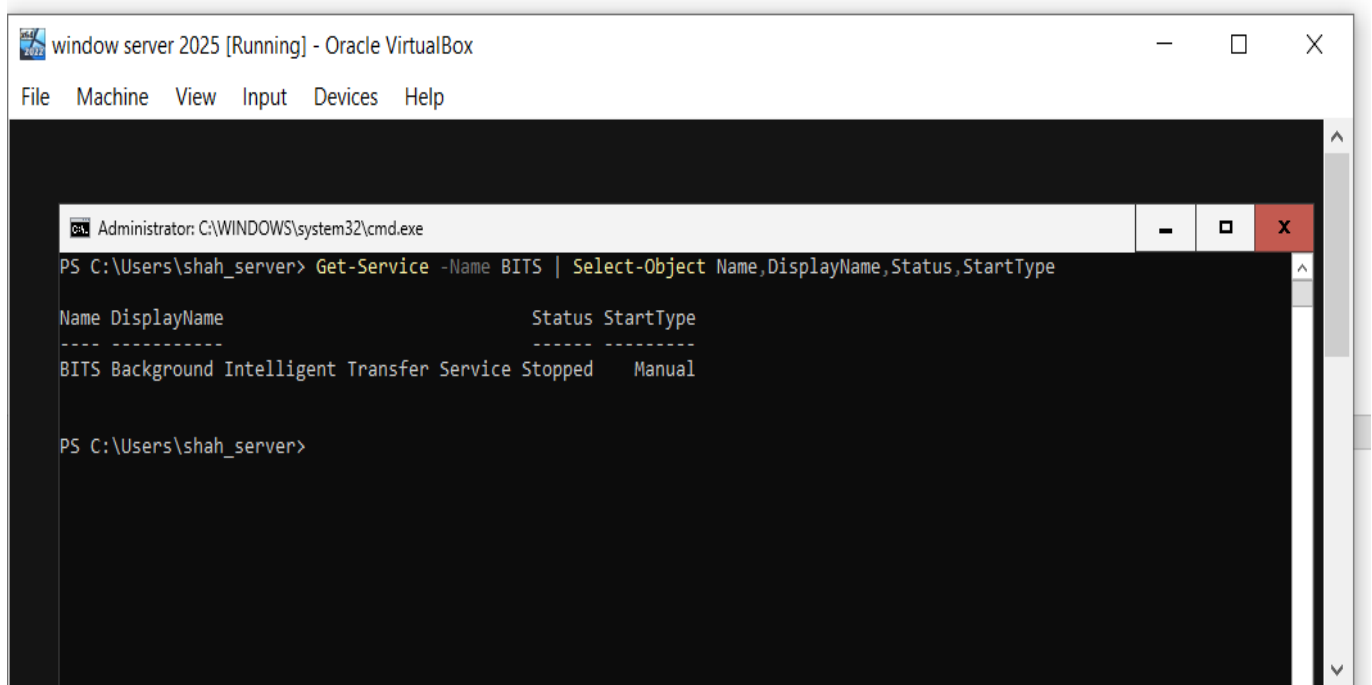


Figure 4. BITS Windows Service status

Screenshot 5 — Running Processes

Command:

```
Get-Process | Sort-Object CPU -Descending | Select-Object -First 10 Name, Id, CPU
```

Purpose:

Demonstrates process monitoring and CPU usage analysis.

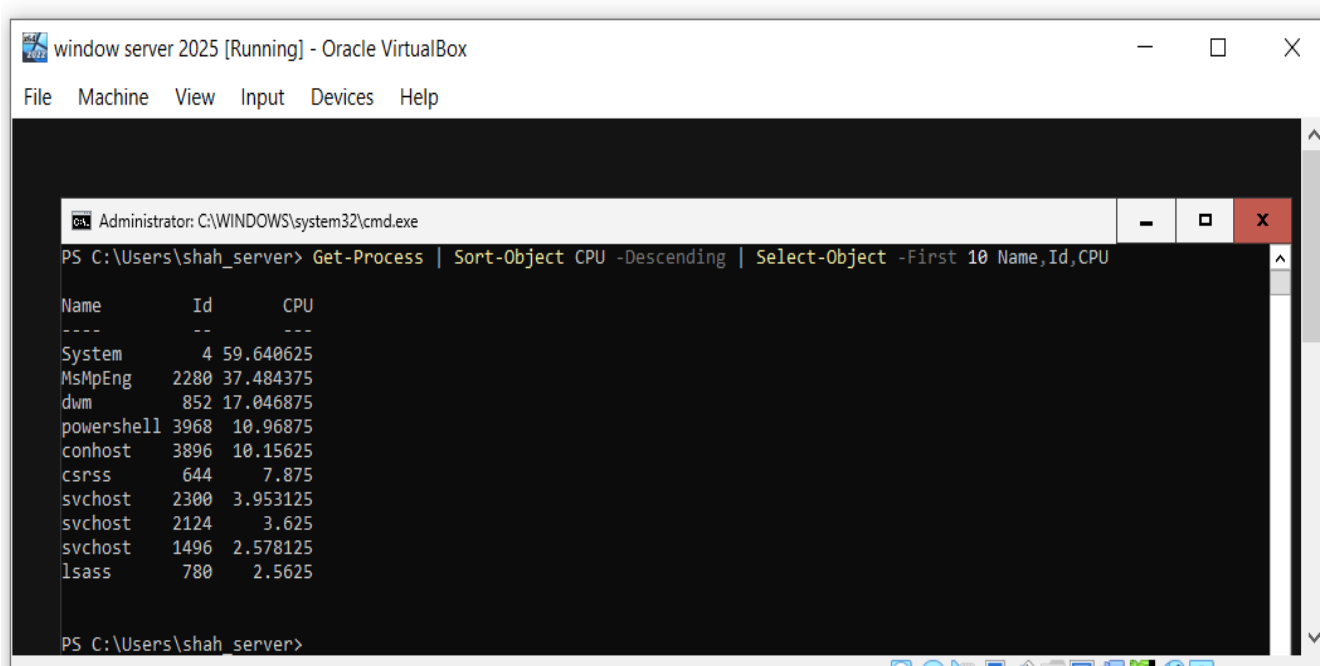


Figure 5. Running Windows processes sorted by CPU usage

Screenshot 6 — Windows Update Inspection

Command:

```
Get-Command *WindowsUpdate*
```

Purpose:

Demonstrates inspection of Windows Update-related PowerShell functionality without installing large updates.

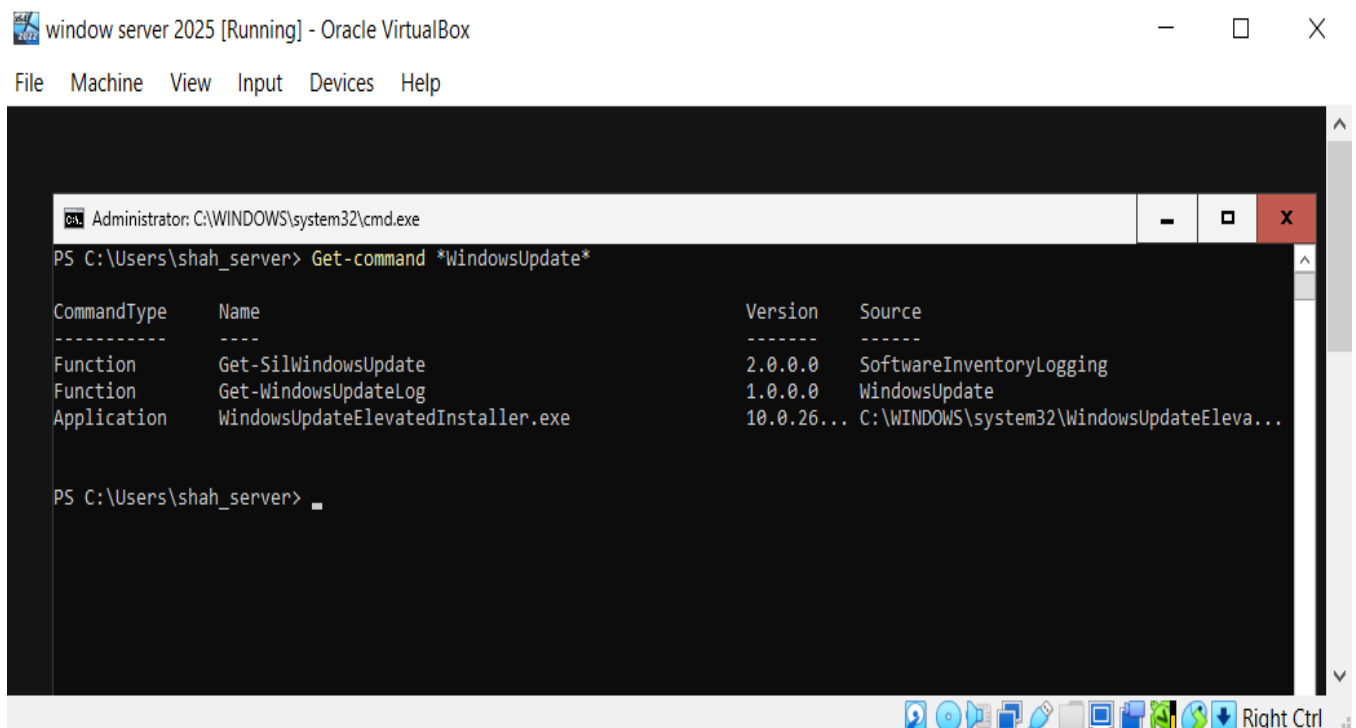


Figure 6. Windows Update PowerShell commands

Screenshot 7 — Lab Directory Structure

Commands:

```
Get-ChildItem "C:\Lab" -Directory
```

and:

```
Get-Acl "C:\Lab" | Select-Object Path, Owner
```

Purpose:

Demonstrates the organized laboratory directory and its ownership.

Expected structure:

```
C:\Lab >>Logs >> Scripts >> Tools
```

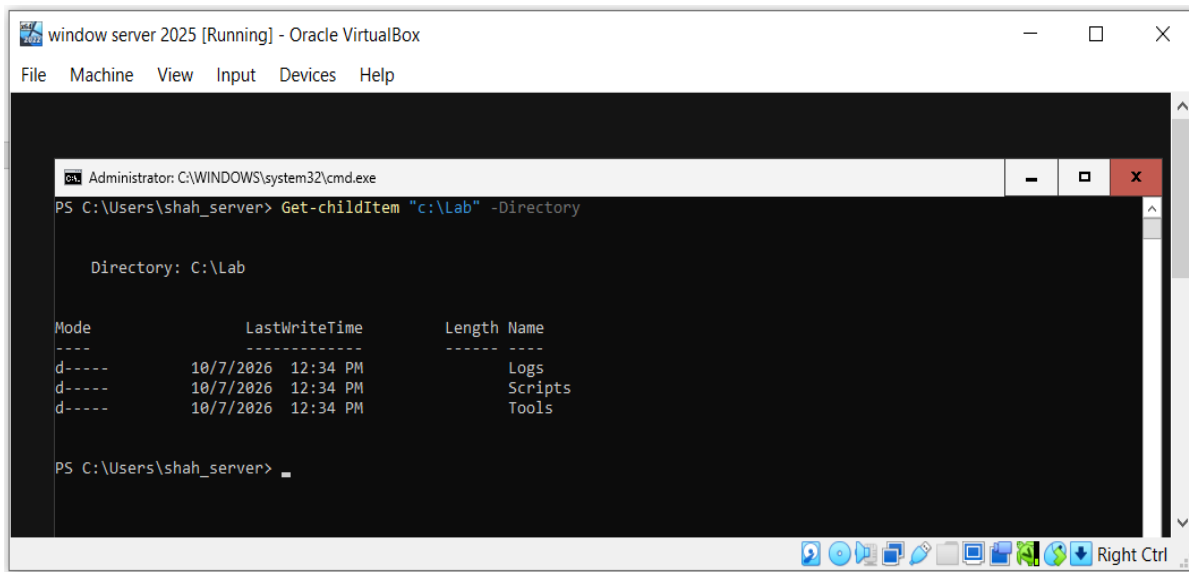


Figure 7. Windows Server laboratory directory structure

36. Project Conclusion

This Windows Server 2025 home lab provided practical experience with Windows system administration and foundational cybersecurity operations.

The project focused on understanding how a Windows Server can be configured, monitored, and secured using PowerShell.

The most important outcome was not only installing Windows Server, but learning how to inspect and manage users, firewall rules, services, processes, logs, network connectivity, and system configuration.

The completed lab will serve as a practical foundation for future cybersecurity projects and can be presented as part of a technical portfolio.